A Formally Verified Provisioning Protocol for Low-Cost IoT Devices

Sana Gul
IMT School for Advanced Studies
Lucca, Italy
sana.gul@imtlucca.it

Matteo Busi
Ca' Foscari University
Venice, Italy
matteo.busi@unive.it

Riccardo Focardi
Ca' Foscari University
Venice, Italy
focardi@unive.it

Flaminia L. Luccio
Ca' Foscari University
Venice, Italy
luccio@unive.it

Francesco Palmarini
10Sec S.r.l.
Venice, Italy
palmarini@10sec.it

***Abstract*—The growth of IoT has increased the need for secure device provisioning. However, many deployments operate under strict cost constraints. In the low-cost segment, dedicated secure elements are often not affordable. At the same time, complex global supply chains enlarge the attack surface and expose devices to cloning, tampering, and credential injection before deployment. This setting requires a careful balance between security and cost. In this paper, we present a lightweight provisioning protocol designed for low-cost IoT devices without secure elements. The protocol is based on a simplified variant of the Noise framework, adapted to reduce computational and implementation overhead while maintaining strong security guarantees. Given the complexity of cryptographic protocol design, we formally model and verify the protocol to establish authentication and key secrecy under a precisely defined attacker model. To the best of our knowledge, this is the first formally verified provisioning protocol specifically targeting low-budget IoT deployments.**

***Index Terms*—IoT Security, Provisioning Protocols, Formal Methods, Lightweight Design.**

I. Introduction

The Internet of Things (IoT) has become a foundational component of modern digital infrastructure, spanning industrial automation, smart homes, logistics, healthcare, and critical infrastructure. A fundamental requirement in these deployments is secure device provisioning, namely the process by which a device is initialized with cryptographic credentials and securely enrolled into its target system. Weaknesses at this stage can permanently compromise devices and enable large scale attacks.

Provisioning security is particularly challenging in the low cost segment of the IoT market. Many devices are produced under strict cost constraints, where adding dedicated secure elements or hardware security modules is economically infeasible. As a result, designers must rely on general purpose microcontrollers with limited computational capabilities and without hardware backed key protection. At the same time, IoT supply chains are increasingly complex and globally distributed. Devices may pass through multiple manufacturing, integration, and distribution stages before reaching their operational environment. This enlarged supply chain significantly increases the attack surface, enabling adversaries to attempt device cloning, firmware modification, or credential extraction prior to deployment.

These constraints create a fundamental tension between security and cost. High assurance solutions typically rely on specialized hardware and complex trust infrastructures, which are incompatible with low budget devices. Conversely, ad hoc lightweight solutions often lack rigorous security analysis and may fail under realistic attacker models. Given the subtle nature of cryptographic protocol design, informal reasoning is insufficient, especially in adversarial supply chain scenarios. Formal verification is therefore essential to obtain strong and well defined security guarantees.

In this work, we address these challenges by designing and formally verifying a minimal and resource-conscious provisioning protocol tailored for low-cost IoT devices without secure elements. Our design is based on a simplified variant of the Noise framework [7], chosen for its clear security goals and modular structure. We adapt and restrict the framework to minimize computational overhead and implementation complexity while preserving strong authentication and key establishment properties.

We formally model and verify the protocol using the Tamarin prover [6]. Tamarin allows us to specify a precise symbolic attacker model with full control over the network and to reason about authentication and secrecy properties in a rigorous manner. Our model captures realistic adversarial capabilities, including supply chain interference prior to deployment. Using Tamarin, we establish authentication and session key secrecy under explicitly formalized trust assumptions, in particular regarding ephemeral key protection and the integrity of the manufacturing channel.

This work is partially supported by projects "SEcurity and RIghts In the CyberSpace - SERICS" (PE00000014 - CUP H73C2200089001), and PRIN/PNRR "Automatic Modelling and ∀erification of Dedicated sEcUrity deviceS - AM∀DEUS" (P2022EPPHM - CUP H53D23008130001), all under the National Recovery and Resilience Plan (NRRP) funded by the European Union - NextGenerationEU.

To the best of our knowledge, this is the first provisioning protocol specifically targeting low budget IoT devices without secure elements that is accompanied by formal verification. Our contribution demonstrates that meaningful security guarantees can be achieved even in cost constrained environments, provided that protocol design and verification are considered jointly.

The remainder of this paper is organized as follows. Section II presents background information together with the system and threat model. Section III describes the provisioning protocol design. Section IV details the formal modeling and verification in Tamarin. Section V concludes the paper.

II. Background and Related Work

A. The Tamarin Prover

To rigorously analyze our protocol, we employ the Tamarin prover [6], a symbolic verification tool for cryptographic protocols. Tamarin models protocol behavior using multiset rewriting rules and evaluates security properties over traces of protocol executions. Both honest participants and the adversary are represented within the same formal framework.

Protocol Specification: A Tamarin model consists of rules of the general form:

```
rule RuleName:
  [ premises ] --[ actions ]-> [ conclusions ]
```

The left-hand side contains *premise facts* that must be present for the rule to apply. The right-hand side introduces *conclusion facts* that describe the updated protocol state. The middle part contains *action facts*, which are recorded in the execution trace and later referenced in security properties.

Facts represent protocol state, knowledge, or events. Tamarin provides built-in facts for communication and freshness, including `In(m)` for message reception, `Out(m)` for message transmission, and `Fr(x)` for the generation of a fresh value unknown to the adversary.

Facts in Tamarin are either *linear* or *persistent*. Linear facts are consumed when a rule that references them is executed, thereby modeling evolving state. Persistent facts, denoted by a leading exclamation mark such as `!Pk(A,pkA)`, which links the public key `pkA` to principal `A`, are not consumed and remain available indefinitely once introduced.

Execution Semantics and Adversary Model: Protocol execution begins from an initially empty multiset of facts. A rule is enabled when its premises match the current state under some substitution of variables. When applied, linear premises are removed, persistent premises remain, action facts are appended to the trace, and conclusion facts are inserted into the state.

The adversary is modeled using Tamarin's built-in Dolev–Yao intruder. The intruder has full control over the communication channel, can intercept and modify messages, and can derive new messages according to the specified equational theory of the employed cryptographic primitives.

Specification of Security Properties: Security goals are expressed as first-order temporal formulas over action facts. These formulas quantify over timepoints in the trace and allow reasoning about ordering and causality of events. Given a specification and a set of lemmas, Tamarin attempts to automatically prove the stated properties. If a property does not hold, the tool produces a counterexample trace illustrating an attack. This trace-based reasoning provides strong assurance that the verified properties hold for all possible executions within the symbolic model.

B. Related Work

The Noise Protocol Framework [7] defines a modular approach to constructing authenticated key exchange and secure channel protocols based on Diffie–Hellman operations. A Noise handshake pattern specifies an ordered sequence of message exchanges together with the associated Diffie–Hellman computations (e.g., static–static, ephemeral–static, ephemeral–ephemeral) used to derive shared secrets.

Noise maintains an internal state consisting of a chaining key and a transcript hash, which are updated as messages are processed. From these values, symmetric encryption keys are derived and used with AEAD to provide confidentiality and authentication. The framework defines a catalog of standard handshake patterns with well-understood security properties under standard assumptions.

Formal verification tools such as ProVerif [2] and the Tamarin Prover [6] are widely used to establish secrecy and authentication properties of cryptographic protocols under symbolic attacker models, and they have been successfully applied to the verification of real-world protocols such as TLS [1], [3].

The formal security of Noise handshake patterns has been analyzed comprehensively by Girol et al. [4], who provided an automated symbolic verification of Diffie–Hellman-based handshake patterns defined in the framework. Similarly, Jacomme et al. [5] conducted a detailed formal analysis of the EDHOC protocol, a lightweight authenticated key exchange standardized for constrained IoT devices, verifying its authentication and secrecy guarantees.

In contrast to these works, which focus on the cryptographic structure of key exchange protocols, our contribution models and verifies a provisioning workflow in which a Noise-inspired handshake is embedded within a manufacturing and key-installation process. By explicitly capturing supply-chain assumptions, device key installation, and cross-manufacturer interference conditions, our analysis extends formal verification beyond the handshake layer to the broader provisioning context.

III. Provisioning Protocol Design

This section presents the design of our lightweight provisioning protocol for low-cost IoT devices without hardware-backed key protection. The protocol is engineered for constrained environments and realistic supply-chain threat assumptions. Here, *lightweight* refers to reduced protocol complexity: two message flights, Diffie–Hellman and AEAD only,

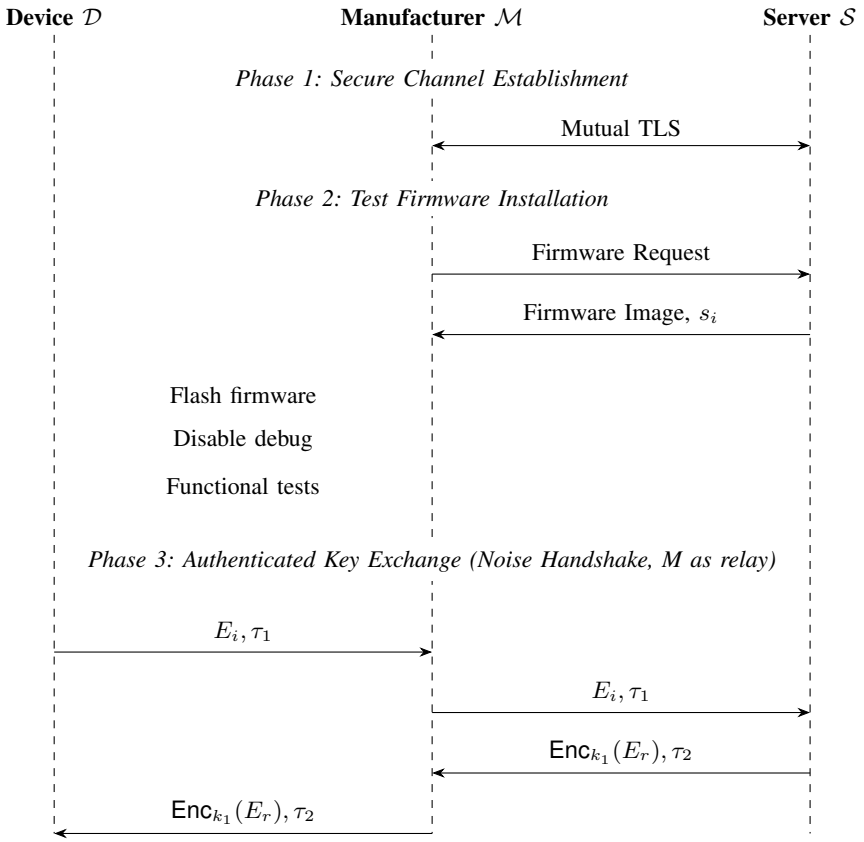


Fig. 1. Provisioning protocol.

no certificate validation on the device, and no explicit Noise transcript state management.

Our primary objective is to enable secure enrollment of freshly manufactured devices into the vendor's infrastructure while preventing credential exposure, device cloning, and unauthorized firmware manipulation, even in the presence of an untrusted manufacturing environment. To achieve this, we decouple manufacturing operations from cryptographic trust establishment. The manufacturer computer is treated as an untrusted relay that facilitates communication but must not gain access to long-term device secrets.

The protocol is structured in three phases: (i) secure channel establishment between the manufacturer computer and the provisioning server, (ii) installation and execution of a minimal provisioning firmware on the device, and (iii) a customized Noise-based authenticated key exchange between the device and the provisioning server, relayed by the manufacturer. The key exchange establishes a confidential session channel through which device-specific credentials are provisioned.

Our design deliberately restricts the Noise framework to a simplified and computationally lightweight pattern while preserving strong mutual authentication and session key secrecy. The resulting construction minimizes implementation complexity on resource-constrained microcontrollers and supports rigorous formal analysis.

A. Actors and Threat Model

The provisioning architecture involves three entities:

- **Provisioning Server ($\mathcal{S}$)**: A trusted server operated by the device vendor. It manages device provisioning, distributes firmware, and injects cryptographic material into devices.
- **Manufacturer Computer ($\mathcal{M}$)**: An untrusted machine located at the manufacturing facility. It connects locally to the device via a debug/programming interface and remotely to the provisioning server via the Internet. $\mathcal{M}$ contains a vendor-provided secure element used to establish a mutually authenticated TLS channel with $\mathcal{S}$.
- **Device ($\mathcal{D}$)**: A freshly assembled device without firmware, credentials, or secret material.

We assume the following threat model and trust assumptions:

- The provisioning server $\mathcal{S}$ is fully trusted.
- The manufacturer computer $\mathcal{M}$ is untrusted and may be malicious.
- Communication between $\mathcal{M}$ and $\mathcal{S}$ is protected by mutually authenticated TLS using credentials stored in the secure element.
- The device $\mathcal{D}$ initially contains no secrets.

Under the modeled assumptions on the TLS channel and firmware handling, the protocol prevents $\mathcal{M}$ from learning long-term device secrets, even though it relays messages between $\mathcal{D}$ and $\mathcal{S}$.

B. Provisioning Workflow

Provisioning proceeds in three phases as depicted in Figure 1.

Phase 1: Secure Channel Establishment. $\mathcal{M}$ establishes a mutually authenticated TLS channel with $\mathcal{S}$ using its hardware-backed credentials. All subsequent communication between $\mathcal{M}$ and $\mathcal{S}$ occurs over this channel.

Phase 2: Test Firmware Installation.

1) $\mathcal{M} \rightarrow \mathcal{S}$: Request provisioning firmware.
2) $\mathcal{S} \rightarrow \mathcal{M}$: Return firmware image. The firmware contains the device long-term private key s_i, which is generated by $\mathcal{S}$ and transmitted exclusively over the mutually authenticated TLS channel.
3) $\mathcal{M}$ flashes the firmware onto $\mathcal{D}$.
4) $\mathcal{M}$ permanently disables the debug interface of $\mathcal{D}$ to prevent further external read/write access to memory.
5) $\mathcal{M}$ restarts $\mathcal{D}$ and performs functional tests.

After debug locking, extraction of s_i via the external interface is no longer possible. The provisioning firmware contains only the minimal logic required to execute the authenticated key exchange.

Phase 3: Authenticated Key Exchange and Secret Injection. $\mathcal{M}$ instructs $\mathcal{S}$ to initiate a Noise-based authenticated key exchange with $\mathcal{D}$. During this phase, $\mathcal{M}$ acts strictly as a relay.

1) $\mathcal{D}$ and $\mathcal{S}$ execute a customized Noise handshake.
2) A shared session key k_{sess} is derived.
3) Over the resulting secure channel, $\mathcal{S}$ provisions additional long-term device credentials.

After successful provisioning, the temporary provisioning logic is erased or permanently disabled. Only a minimal bootloader responsible for loading the production firmware remains active.

C. Customized Noise-Based Handshake

We use the following cryptographic notation. The protocol relies on Diffie–Hellman key exchange over a cyclic group generated by G.

Lowercase symbols denote private keys and uppercase symbols denote public keys. Static key pairs are written (s_i, S_i) and (s_r, S_r) for initiator and responder, respectively. Ephemeral key pairs (e_i, E_i) and (e_r, E_r) are freshly generated for each execution and erased after use.

Given a private key x and public key Y, we denote the Diffie–Hellman computation by $\mathsf{DH}(x, Y)$. We assume the standard correctness property:

$$\mathsf{DH}(x, Y) = \mathsf{DH}(y, X)$$

whenever $X = xG$ and $Y = yG$.

Key derivation is performed using a hash-based key derivation function $\mathsf{HKDF}(\cdot)$. Authenticated encryption is denoted by $\mathsf{AEAD_Enc}_k(m)$ and decryption/verification by $\mathsf{AEAD_Dec}_k(c)$.

At the beginning of the handshake, both parties possess their own static key pairs. The device is provisioned with s_i and S_r during Phase 2.

Message 1 (Initiator → Responder).

$$\begin{aligned}(e_i, E_i) &\leftarrow \mathsf{GenEphemeral()}\\ ES &= \mathsf{DH}(e_i, S_r)\\ SS &= \mathsf{DH}(s_i, S_r)\\ k_1 &= \mathsf{HKDF}(ES \| SS)\\ \tau_1 &= \mathsf{AEAD_Enc}_{k_1}(\epsilon)\end{aligned}$$

The initiator sends (E_i, τ_1).

Message 2 (Responder → Initiator). Upon receiving E_i, the responder computes:

$$\begin{aligned}ES &= \mathsf{DH}(s_r, E_i)\\ SS &= \mathsf{DH}(s_r, S_i)\\ k_1 &= \mathsf{HKDF}(ES \| SS)\end{aligned}$$

If $\mathsf{AEAD_Dec}_{k_1}(\tau_1)$ is successful, the responder proceeds:

$$\begin{aligned}(e_r, E_r) &\leftarrow \mathsf{GenEphemeral()}\\ EE &= \mathsf{DH}(e_r, E_i)\\ SE &= \mathsf{DH}(e_r, S_i)\\ k_2 &= \mathsf{HKDF}(k_1 \| EE \| SE)\end{aligned}$$

Thus, k_2 depends on the previously derived key material k_1, capturing the incremental nature of the key schedule. The responder sends:

$$\mathsf{AEAD_Enc}_{k_1}(E_r), \quad \tau_2 = \mathsf{AEAD_Enc}_{k_2}(\epsilon).$$

Finalization. The initiator decrypts and verifies the encrypted E_r, recomputes EE and SE, derives k_2, and verifies τ_2. Upon successful verification, both parties derive the session key $k_{\text{sess}} = k_2$.

Our construction is inspired by the Noise KK pattern, in which the initiator knows the responder's static public key in advance, but deliberately simplifies the standard Noise state machine. In particular, we omit the explicit transcript hash and chaining key variables and instead employ an incremental key derivation process in which successive Diffie–Hellman outputs are incorporated through HKDF invocations. Authentication is achieved via explicit AEAD verification rather than implicit transcript binding.

Furthermore, static public keys are provisioned out-of-band during the manufacturing phase and are therefore not transmitted as part of the handshake itself. These design choices reduce implementation complexity and state management requirements while preserving the mutual authentication and session key establishment properties required in our setting.

```
rule S_Register_Static:
  [ Fr(~sr) ]
-->
  [ !ServerLtk($S, ~sr),
    !ServerLtPk($S, 'g'^~sr) ]
```

Listing 1. Server S key pair generation

```
rule Init_TLS:
  [ !ServerLtPk(S, srp),
    Fr(~si) ]
-->
  [ TLS_Establish($M, S, $D, ~si, srp) ]

rule TLS_OK:
  [ TLS_Establish(M, S, D, si, srp) ]
-->
  [ !TLS_payload(M, S, D, si, srp),
    !DeviceLtk(D, si),
    DeviceReady(D, M, S, si, srp),
    ServerReady(S, M, D, 'g'^si, srp)
  ]
```

Listing 2. Abstract TLS installation of device D LTK

```
rule D_Init:
  let
    eip = 'g'^~ei
    ES = srp^~ei
    SS = srp^si
    k1 = h(<'HKDF_1', ES, SS>)
    t1 = senc('empty',k1)
  in
  [ Fr(~ei),
    DeviceReady($D, $M, $S, si, srp) ]
-->
  [ Out(<eip, t1>),
    St_I($D, $M, $S, ~ei, si, k1, srp),
    !Eph_Device($D, ~ei) ]
```

Listing 3. Device D initiates the Noise-like handshake

```
rule D_Response_Init:
  let
    EE = Er^~ei
    SE = Er^~si
    k2 = h(<'HKDF_2', k1, EE, SE>)
  in
  [ In(<senc(<'k1',Er>,k1),
       senc('empty',k2)>),
    St_I($D, $M, $S, ~ei, ~si, k1, srp)
  ]
-->
  [ Secret_K2_Device($D, $S, $M, k2),
    St_Device_Ready($D, $M, $S, k2) ]
```

Listing 4. Device D finalises and obtains k2

Fig. 2. A selection of Tamarin snippets

IV. Formal model and results

We modelled the provisioning protocol in the Tamarin prover using a small symbolic theory supporting Diffie–Hellman, symmetric encryption and hashing. The model focuses on the provisioning handshake (a simplified Noise-like two-message pattern) and an abstracted `TLS` phase used to convey the device long-term secret into the device during manufacturing. We do not represent the full TLS handshake: instead we represent the effect of a mutually-authenticated channel by facts that capture payload availability (`!TLS_payload`) and the installation of the device long-term key (`!DeviceLtk`). The model allows both passive leaks of the TLS payload (`TLSleak`) and an abstract integrity break (`TLSinject`); lemmas are stated under assumptions that exclude these behaviours when required.

Below we summarise the main modeling choices and show the most relevant Tamarin fragments used in proofs.

A. Modelling highlights

- **Key material.** The server long-term keypair is generated by a single rule that creates both the private scalar and the public group element. The device long-term secret is generated/installed during the abstracted TLS phase and then used in the Noise-like handshake.
- **Noise-like handshake.** The initiator (the device) generates an ephemeral secret e_i and uses the server static public key S_r together with its own static secret s_i to derive the first key material k_1. The responder (server) then replies with its ephemeral E_r encrypted under k_1 and provides key confirmation in a second AEAD under the derived k_2. This mirrors the KK style structure but is simplified for constrained devices.
- **Adversary model.** Tamarin's Dolev–Yao intruder controls the network. We provide dedicated rules for revealing long-term keys and ephemerals to model compromise. TLS compromises are modelled via two rules: a passive leak (`TLSleak`) and an integrity/binding break (`TLSinject`). Security claims are made conditional on excluding these compromises where appropriate.

The snippets in Figure 2 reproduce the core rules of the model and correspond exactly to the rules used in the formal proofs.

B. Verification Results

We verified protocol executability, session-key secrecy, authentication, and secrecy of the provisioned device secret under a range of compromise assumptions. The analysis highlights which trust assumptions are essential for security and which compromises lead to concrete attacks.

Device compromise. Device compromise is modeled by the rule `RevealLtk(Device)`, which releases the device long-term secret s_i to the adversary. Under this compromise, both session-key secrecy and authentication attacks become possible in the unconstrained model. More precisely:

- **Session-key secrecy.** If the device long-term key is revealed and the device ephemeral secret is also revealed

(`RevealEph(Device)`), the adversary can derive the session key k_2. Session-key secrecy is therefore restored by explicitly excluding ephemeral compromise:

```
not (Ex #j. RevealEph(Device) @ j)
```

- **Authentication and provisioning secrecy.** If, in addition to device compromise, the abstract TLS integrity break (`TLSinject`) is permitted, the adversary can mount authentication attacks by injecting malicious ready states. Authentication and provisioning secrecy are restored by excluding TLS integrity violations:

```
not (Ex M D #j. TLSinject(M,Server,D) @ j)
```

Thus, device long-term compromise alone does not invalidate the claims provided that ephemeral secrets remain protected and TLS integrity is maintained.

Server compromise. Server compromise is modeled by `RevealLtk(Server)`, which releases the server's long-term private key s_r.

In this case, no attacks are possible as long as the device long-term secret remains confidential. However, if the device public key (equivalently, the device long-term secret) is also leaked, both session-key secrecy and authentication properties can be violated. This reflects the fact that the handshake includes a static–static Diffie–Hellman component, so compromising both static keys allows reconstruction of the derived key material.

TLS payload leakage. The rule `TLSleak` models passive disclosure of the TLS payload used to install the device long-term secret. When this leakage is allowed, authentication attacks become possible, in particular cross-manufacturer exploits where one manufacturer can interfere with another device's provisioning flow.

These attacks are prevented by excluding both TLS integrity violations and device-ephemeral compromise:

```
not (Ex M D #j. TLSinject(M,Server,D) @ j)
not (Ex #j. RevealEph(Device) @ j)
```

Summary of trust assumptions. The formal analysis therefore shows that the protocol achieves:

- Session-key secrecy, provided that device ephemerals are not revealed and TLS integrity is preserved.
- Authentication and provisioning secrecy, provided that TLS integrity is preserved and the device ephemeral secret remains confidential.

The results make explicit the minimal trust boundary required for secure provisioning: protection of ephemeral device secrets during execution and integrity of the TLS channel used during manufacturing. Under these assumptions, the model establishes confidentiality of the provisioned secret and agreement between device and server on successful completion.

Reproducibility. The Tamarin source files used for verification are provided as online supplementary material to this paper: https://sites.google.com/view/csr26-tamarin-provisioning

V. Conclusion

This paper presented a lightweight provisioning protocol designed for low-cost IoT devices without dedicated secure elements and provided a formal verification using the Tamarin prover. The formal model allowed us to make all security assumptions explicit and to precisely characterize the conditions under which session-key secrecy, authentication, and provisioning secrecy hold.

The analysis confirms that the protocol achieves its intended guarantees provided that (i) device ephemeral secrets are not disclosed during execution and (ii) the integrity of the TLS channel used during manufacturing is preserved. The model also clarifies the impact of long-term key compromise on both device and server, and shows how certain attacks arise only when multiple compromise conditions are combined.

A central outcome of the verification effort is the identification of the critical trust boundary at the manufacturing stage. Although the manufacturer should be generally considered untrusted with respect to network behavior, the security properties rely on the assumption that firmware installation and key handling are not maliciously altered outside the modeled TLS channel. In practice, this highlights the importance of protecting provisioning firmware and preventing unauthorized manipulation of device keys during production.

Furthermore, the formal analysis revealed that some attack traces can only be excluded by ruling out cross-manufacturer interference, modeled by the `TLSinject` rule. In complex supply-chain scenarios involving multiple production sites or subcontractors, such cross-domain interactions may represent a realistic risk. The need to explicitly exclude these behaviors demonstrates the value of formal modeling in uncovering subtle dependency conditions that may otherwise remain implicit.

Overall, our results show that meaningful security guarantees are achievable in cost-constrained IoT environments under enforced manufacturing integrity and ephemeral key protection. Future work will focus on reducing manufacturing-time trust and strengthening resilience against cross-domain supply-chain threats.

References

[1] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. Verified models and reference implementations for the TLS 1.3 standard candidate. In *IEEE Symposium on Security and Privacy*, 2017.

[2] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016.

[3] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM CCS*, 2017.

[4] Guillaume Girol, Lucca Hirschi, Ralf Sasse, Dennis Jackson, Cas Cremers, and David Basin. A Spectral Analysis of Noise: A Comprehensive, Automated, Formal Analysis of Diffie-Hellman Protocols. In *29th USENIX Security Symp.*, 2020.

[5] Charlie Jacomme, Elise Klein, Steve Kremer, and Maïwenn Racouchot. A comprehensive, formal and automated analysis of the EDHOC protocol. In *32nd USENIX Security Symposium*, 2023.

[6] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The Tamarin prover for the symbolic analysis of security protocols. In *CAV*, volume 8044 of *LNCS*, pages 696–701. Springer, 2013.

[7] Trevor Perrin. The Noise Protocol Framework. https://noiseprotocol.org/noise.pdf, 2018. Accessed: March 2026.